

3. AI Large Model Voice Interaction

3. AI Large Model Voice Interaction

1. Concept Introduction

1.1 What is "AI Large Model Voice Interaction"?

1.2 Brief Description of the Principle

2. Code Analysis

Key Code

1. Voice Input Node (`targetmodel/targetmodel/asr.py`)

2. AI Service and Voice Output Node (`targetmodel/targetmodel/model_service.py`)

Code Analysis

3. Practical Operation

3.1 Configure Offline Voice Interaction

3.2 Start and Test the Function

Note: The RDK X5 4GB version cannot run this offline due to performance limitations. Please refer to the tutorial for online large models.

1. Concept Introduction

1.1 What is "AI Large Model Voice Interaction"?

In the `targetmodel` project, **AI Large Model Voice Interaction** connects the previously introduced **Offline ASR** and **Offline TTS** with the **Large Language Model (LLM)** core to form a complete conversational system that can listen, speak, and think.

This is no longer an isolated function, but the prototype of a true **voice assistant**. Users can have natural language conversations with the robot through voice, and the robot can understand questions, think of answers, and respond with voice. The entire process is completed locally without the need for a network.

The core of this function is the `model_service` **ROS2 node**. It acts as the brain and nerve center, subscribing to the recognition results of ASR, calling the LLM to think, and then publishing the LLM's text reply to the TTS node for speech synthesis.

1.2 Brief Description of the Principle

The implementation principle of this function is a classic data flow pipeline:

1. **Sound -> Text (ASR):** The `asr` node continuously listens to the ambient sound. Once it detects that the user has finished speaking a sentence, it converts it into text and publishes it to the `/asr_text` topic.
2. **Text -> Thinking -> Text (LLM):** The `model_service` node subscribes to the `/asr_text` topic. When it receives the text from ASR, it sends it as a prompt to the locally deployed large language model (such as `Qwen` run through `ollama`). The LLM will generate a text response based on the context.
3. **Text -> Sound (TTS):** After the `model_service` node gets the LLM's response, it publishes it to the `/tts_text` topic.

4. **Sound Playback:** The `tts_only` node subscribes to the `/tts_text` topic. After receiving the text, it immediately calls the offline TTS model to synthesize it into audio and play it through the speaker.

This process forms a complete closed loop of **voice input -> text processing -> text output -> voice output**.

2. Code Analysis

Key Code

1. Voice Input Node (`largemodel/largemodel/asr.py`)

```
# From largemodel/largemodel/asr.py
class ASRNode(Node):
    def __init__(self):
        # ...
        self.asr_pub = self.create_publisher(String, "asr", 5)
        # ...

    def kws_handler(self)->None:
        if self.listen_for_speech(self.mic_index):
            asr_text = self.ASR_conversion(self.user_speechdir)
            if asr_text != 'error':
                self.asr_pub_result(asr_text)

    def asr_pub_result(self, asr_result:str)->None:
        msg=String(data=asr_result)
        self.asr_pub.publish(msg)
```

2. AI Service and Voice Output Node

(`largemodel/largemodel/model_service.py`)

```
# From largemodel/largemodel/model_service.py
class LargeModelService(Node):
    def __init__(self):
        # ...
        self.asrsub = self.create_subscription(String, 'asr', self.asr_callback, 1)
        # ...

    def asr_callback(self, msg):
        """Callback function for handling ASR messages."""
        # ...
        result = self.model_client.infer_with_text(msg.data,
        message=messages_to_use)
        self.process_model_result(result)

    def process_model_result(self, result, from_seewhat=False):
        """Process the result returned by the model."""
        # ...
        user_friendly_response = response_json.get("response", "I'm
processing...")
        # ...
```

```

        self._safe_play_audio(user_friendly_response)
        # ...
        self.execute_tools(tools_list)

def _safe_play_audio(self, text_to_speak: str):
    """
    Synthesizes and plays all non-empty messages only in non-text chat mode.
    """
    if not self.text_chat_mode and text_to_speak:
        try:
            self.model_client.voice_synthesis(text_to_speak,
self.tts_out_path)
            self.play_audio_async(self.tts_out_path)
        except Exception as e:
            self.get_logger().error(f"Safe audio playback failed: {e}")

```

Code Analysis

The voice interaction function of the AI large model is jointly completed by two independent ROS nodes, `asr.py` and `model_service.py`. They communicate with each other through the ROS topic `/asr` to form a complete processing loop.

1. Voice Input and Publishing (`asr.py`):

- The function of the `ASRNode` node is to process voice input. In the `kws_handler` function, it completes recording and text conversion.
- After successful conversion, the `asr_pub_result` function is called. This function puts the obtained text string into a `std_msgs.msg.String` message, and then publishes the message to the `/asr` topic through the `self.asr_pub` publisher.

2. Subscription and Response (`model_service.py`):

- When the `LargeModelService` node is initialized, it subscribes to the `/asr` topic through the `create_subscription` method and specifies `asr_callback` as its callback function.
- When `ASRNode` publishes a message on the `/asr` topic, the ROS system will automatically call the `asr_callback` function of `LargeModelService` and pass the message as a parameter `msg`.

3. Interaction Loop:

- After the `asr_callback` function receives the message, it extracts the text data (`msg.data`) and passes it to the large model for inference (`self.model_client.infer_with_text`).
- After the inference is completed, the `process_model_result` function is called. This function extracts the text for reply from the model result.
- Then, the `_safe_play_audio` function is called, which uses the TTS function (`self.model_client.voice_synthesis`) to synthesize the reply text into audio and play it.
- At this point, a complete interaction loop from "receiving user voice" to "responding to the user with voice" is completed. The two nodes are decoupled through the publish-subscribe mode of ROS. `asr.py` is responsible for input, and `model_service.py` is responsible for processing and output.

3. Practical Operation

3.1 Configure Offline Voice Interaction

To implement a fully offline voice interaction system, you need to ensure that the ASR, TTS, and LLM parts are all configured in offline mode.

1. Open the main configuration file `yahboom.yaml`:

```
vim ~/yahboom_ws/src/largemodel/config/yahboom.yaml
```

2. Modify/Confirm the following key configurations:

```
asr:
  ros__parameters:
    use_online_asr: False           # Key: Set to False to enable offline
    ASR
    regional_setting : "China"

model_service:
  ros__parameters:
    use_ollinetts: False           # Key: Set to False to enable
    offline TTS
    llm_platform: 'ollama'         # Key: Set to 'ollama' to enable
    offline LLM
    regional_setting : "China"
```

3. Open the model interface configuration file `large_model_interface.yaml`:

```
vim ~/yahboom_ws/src/largemodel/config/large_model_interface.yaml
```

4. Confirm that all offline model paths are correct:

```
# large_model_interface.yaml

## Offline Large Model
ollama_model: "qwen2.5:7b" # Make sure this model has been downloaded via
ollama pull

## Offline Speech Recognition
local_asr_model: "~/yahboom_ws/src/largemodel/MODELS/asr/SenseVoicesSmall" #
Make sure the ASR model path is correct

## Offline TTS
# Chinese TTS model
zh_tts_model: "/home/sunrise/yahboom_ws/src/largemodel/MODELS/tts/zh/zh_CN-
huayan-medium.onnx"
zh_tts_json: "/home/sunrise/yahboom_ws/src/largemodel/MODELS/tts/zh/zh_CN-
huayan-medium.onnx.json"

# English TTS model
en_tts_model: "/home/sunrise/yahboom_ws/src/largemodel/MODELS/tts/en/en_US-
libritts-high.onnx"
```

```
en_tts_json: "/home/sunrise/yahboom_ws/src/largemodel/MODELS/tts/en/en_US-  
libritts-high.onnx.json"
```

```
# Make sure the TTS model path is correct!
```

3.2 Start and Test the Function

Note: Using a model with a small number of parameters or running with limited memory will result in poor performance. For a better experience, please refer to the corresponding chapter in <Online Large Model (Text Interaction)>.

1. Start the `largemodel` main program:

Run the following command to start voice interaction:

```
ros2 launch largemodel largemodel_control.launch.py
```

2. Test:

- **Wake up:** Say to the microphone: "Hi,Yahboom."
- **Conversation:** After the speaker responds, you can say what you want to ask.
- **Observe the log:** In the terminal where the `launch` file is running, you should see:
 1. The ASR node recognizes your question and prints it out.
 2. The `model_service` node receives the text, calls the LLM, and prints the LLM's reply.
- **Listen to the answer:** After a while, you should be able to hear the robot answer your question in a synthesized voice from the speaker.